

Vector Database Algorithm

Imagine you have **1 million vectors** (embeddings of products, text, images, etc.).
If a query comes, comparing with all 1 million vectors is **too slow**.

IVF solves this by **grouping vectors into clusters first**, so you only search in a small part.

Phase 1: Indexing (One-Time Setup)

This is done **offline before search starts**.

Step 1: Find “Centers” (Centroids)

- Database looks at all vectors.
- It creates **100 center points (centroids)**.
- Each centroid represents a region in vector space.

Think of centroids as **group leaders**.

Example analogy:

- Clothes store
- One centroid = Men’s casual shirts
- Another centroid = Women’s dresses
- Another centroid = Shoes

Step 2: Assign Vectors to Clusters

Now every vector is assigned to **nearest centroid**.

So:

- If vector is closest to centroid #42
It goes into **cluster 42**

This is like grouping students into classrooms.

Result After Indexing

Instead of 1 million vectors in one list, you now have:

- Cluster 1 → 10,000 vectors
- Cluster 2 → 9,800 vectors
- Cluster 42 → 10,200 vectors
- ...

Phase 2: Searching (When User Sends Query)

Now the real magic happens.

Step 3: Compare Query with Centroids Only

User query:

“light blue summer shirt”

Instead of comparing with **1 million vectors**,
database compares query with **100 centroids only**.

Finds nearest centroid = **#42 (men's casual shirts)**

Step 4: Search Only Inside That Cluster

Now database searches only in **cluster 42**
which has maybe **10,000 vectors instead of 1,000,000**

HUGE SPEEDUP

Why It's Called “Inverted File Index”

Traditional file:

vector → cluster

IVF stores:

cluster → list of vectors

So the index is **inverted**.

Example Diagram in Image

At bottom of image:

- Red dots = vectors
- Black cross lines = space division
- Clusters form around groups of points

Query vector goes near one cluster, so search happens there.

Time Complexity Difference

Method	Comparisons
Brute force	1,000,000
IVF (100 clusters)	100 + 10,000

100× faster (or more)

Drawback of IVF

Because you **don't search everywhere**, you might miss some nearest vectors.

So IVF is **approximate nearest neighbor (ANN)**.

Trade-off:

- Speed ↑
- Accuracy slightly ↓

Real Systems Using IVF

- FAISS (Facebook AI)
- Pinecone
- Milvus
- Weaviate
- Chroma
- Elasticsearch vector search

Simple Real-Life Analogy

Library with 1 million books.

Without IVF:

You search every book 🤖

With IVF:

1. Go to section "Computer Science"
2. Then search only inside that shelf

IVF = First cluster vectors, then search only in relevant cluster instead of entire database.

Drawbacks of IVF (Inverted File Index)

1. Not 100% Accurate

IVF does **not search all vectors**.

It searches only some clusters.

So it can **miss the true nearest result**.

2. Wrong Clustering

Sometimes vectors go to the **wrong cluster**.

Then the query will not look there → **wrong answer**.

3. Hard to Tune

You must choose:

- Number of clusters
- How many clusters to search

If wrong values:

- Speed ↓
- Accuracy ↓

4. Slow to Build Index

To create clusters, it uses **K-means** on millions of vectors.

This takes a lot of time and memory.

5. Updates Are Difficult

When new data comes:

- Need to assign it to clusters
- Sometimes need to rebuild clusters

Not good for frequently changing data.

6. Extra Memory

IVF stores:

- Centroids

- Cluster lists

Uses more memory than simple brute force.

7. Not Good for Small Data

If dataset is small, brute force is faster.

IVF overhead is useless.

Method = 2

Instead of clustering like IVF,
we **keep cutting the space into halves again and again**.

This creates a **tree structure**.

What Is Binary Space Partitioning?

Binary = split into 2

Space = vector space

Partitioning = dividing

So we **divide space into left/right, up/down, again and again**.

Step-by-Step Explanation

Step 1: Pick a Dimension & Split

We pick one dimension (like x-axis).

Find middle value (median).

All points:

- Left → go to left branch
- Right → go to right branch

Step 2: Split Again (Alternate Dimension)

Now for each group:

- Pick next dimension (y-axis)
- Split again into two halves

Step 3: Keep Splitting

Repeat:

- $x \rightarrow y \rightarrow x \rightarrow y \rightarrow x \rightarrow y$

Until each region has very few points.

This forms a **tree**.

Example with Numbers

Array:

1 2 3 4 5 6 7

Median = 4

So tree:

```
      4
     / \
    2   6
   /\  /\
  1 3 5 7
```

This is exactly **binary search tree** idea.

For Vectors (2D Example in Image)

Points like:

(2,7), (4,5), (6,11) ...

Tree splits:

1. Split by x median
2. Then split by y median
3. Then x again

How Searching Works

When query comes:

1. Go down tree like binary search
2. Reach closest region
3. Check only nearby nodes

Not all points.

Why It Is Fast

Instead of searching all points:

- Search only relevant branches
- Tree depth = $\log(N)$

Much faster than brute force.

Drawbacks (Very Important)

1. Curse of Dimensionality

KD-Tree works well in **low dimensions (2D, 3D)**.

But embeddings have **768D or 1536D**.

Tree becomes useless.

2. Unbalanced Tree Problem

If data is skewed → tree becomes unbalanced → slow.

3. Insert/Delete Hard

Adding new points can break tree balance.

Rebuilding needed.

4. High Memory

Stores tree nodes, splits, pointers.

5. Not Best for Modern Vector DB

For large high-dimensional data, HNSW is better.

Simple Analogy

Imagine land divided:

1. Divide India into North/South
2. Then states
3. Then districts
4. Then villages

To find a house, you go step-by-step instead of checking every house.

Binary space partitioning (KD-tree) recursively splits the vector space into two halves using median values, forming a tree that allows faster nearest neighbor search, but it performs poorly in high-dimensional spaces.

Compare IVF vs KD-Tree

Feature	IVF	KD-Tree
Idea	Clustering	Space splitting
Data Dim	High Dim	Low Dim
Speed	High	Medium
Modern Use	Yes	Rare
Accuracy	Approx	Exact-ish

Method = 3

Instead of clusters or trees,
HNSW makes a **graph like social network**.

Vectors are connected like friends.

What Is HNSW?

HNSW = Hierarchical Navigable Small World

- Hierarchical = multiple layers
- Small world = graph where every node is connected to few neighbors

Structure in Image

You see **3 layers**:

- Layer 2 (top) → very few nodes
- Layer 1 → medium nodes
- Layer 0 (bottom) → all nodes

Top layers = highways

Bottom layer = local roads

Layer 0 (Base Layer)

- Contains **all vectors**
- Each vector connected to nearest neighbors (like friends)

Layer 1 (Middle Layer)

- Only some important nodes
- Fewer connections
- Helps to jump fast

Layer 2 (Top Layer)

- Very few nodes
- Used to quickly jump to correct region

How Search Works (Simple)

Step 1: Start from Top Layer

Query starts from a random node in **layer 2**.

Step 2: Greedy Search

Move to neighbor that is **closer to query**.

Like Google Maps choosing nearest road.

Step 3: Go Down Layers

When no closer node found:

Go to layer 1
Then layer 0

Step 4: Local Search

At layer 0, search neighbors deeply → find nearest vectors.

Why HNSW Is SUPER FAST

Because:

- Long jump in top layers
- Local search in bottom layer
- Graph navigation is logarithmic

Faster than IVF and KD-tree.

Alpha Value in Image

You see:

- $\alpha = 3$ (upper layer connections)
- $\alpha = 6$ (lower layer connections)

Means:

- Top layers have few neighbors
- Bottom layers have more neighbors

Drawbacks of HNSW

1.High Memory

Stores many edges (connections).

More memory than IVF.

2.Hard to Build

Inserting nodes requires graph updates.

3.Hard to Delete

Deletion is complex.

4.Randomness

Graph depends on random insertion order.

Why Companies Love HNSW

- Very high accuracy
- Very fast
- Works great for high-dimensional embeddings

Used in:

- Pinecone
- Weaviate
- Milvus
- Qdrant
- OpenSearch

Simple Analogy

Imagine:

- Top layer = flights
- Middle layer = highways
- Bottom layer = city roads

To reach house:

1. Fly to city
2. Drive on highway
3. Walk in streets

HNSW builds a multi-layer graph where vectors connect to nearby neighbors, enabling fast approximate nearest neighbor search by navigating from top layers to bottom layers.

IVF vs KD-Tree vs HNSW

Method	Idea	Best For
IVF	Clusters	Large static data
KD-Tree	Space split	Low dimension
HNSW	Graph	High dimension (modern)